



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.6: Post-Quantum Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

Why Go Post-Quantum?

The quantum threat

- **Quantum computers** are fundamentally different from classical computers
- They exploit quantum mechanical phenomena: superposition and entanglement
- Can solve certain problems exponentially faster than classical computers
- **Critical insight:** Many cryptographic algorithms rely on problems that quantum computers can solve efficiently

How quantum computers work

- **Classical bits:** Can be either 0 or 1
- **Quantum bits (qubits):** Can be in *superposition* of 0 and 1 simultaneously
- **Superposition:** A qubit can represent both states at once
 - n classical bits: Store one of 2^n possible values
 - n qubits: Can represent all 2^n values simultaneously
- **Entanglement:** Qubits can be correlated in ways impossible classically
 - Measuring one qubit instantly affects others
 - Enables parallel computation across exponentially many states
- **Quantum parallelism:** Process all possibilities at once, extract answer through interference

Building quantum computers: the engineering challenge

- **Multiple approaches being pursued:**
 - Superconducting circuits (IBM, Google)
 - Trapped ions (IonQ, Honeywell)
 - Topological qubits (Microsoft)
 - Photonic qubits (Xanadu, PsiQuantum)
 - Neutral atoms (QuEra, Pasqal)
- **Common requirement:** Maintain quantum coherence
 - Qubits must remain in superposition
 - Any interaction with environment causes *decoherence*
 - Current coherence times: microseconds to milliseconds

Physical challenges: Why qubits are so difficult

- **Size requirements:** Qubits are difficult to physically manipulate
 - Individual photons, electrons, or atomic nuclei
 - Some approaches (superconducting) have larger qubits
 - Impossible to manipulate with conventional tools
 - Require laser beams or electromagnetic fields for control
- **Temperature requirements:** Near absolute zero (-273°C)
 - For some quantum systems, tiny amounts of heat destroy quantum states
 - Requires dilution refrigerators costing millions
 - More cooling power needed as system scales
- **Isolation from environment:**
 - Magnetic fields, vibrations, cosmic rays all cause errors
 - Need multiple layers of shielding
 - Even observation destroys quantum states!

The coherence problem: Racing against time

- **Coherence time:** How long qubits maintain their quantum state
 - Current record: A few seconds at best
 - Most systems: Microseconds to milliseconds
 - Breaking crypto needs: Days to weeks of computation
- **Error rates:** Quantum operations are inherently noisy
 - Classical computers: 1 error per 10^{17} operations
 - Current quantum computers: 1 error per 10^3 operations
 - Need massive improvement to be useful
- **Quantum error correction:**
 - Can't copy quantum states (no-cloning theorem)
 - Need 100-1000 physical qubits to make 1 logical qubit
 - Turns "millions of qubits" into "billions of physical qubits"

What makes current cryptography vulnerable?

- **RSA**: Based on factoring large integers
- **Diffie-Hellman**: Based on discrete logarithm problem
- **Elliptic Curve Cryptography**: Also based on discrete logarithm problem
- These are *hard* for classical computers but *easy* for quantum computers
- **Shor's algorithm** (1994): Polynomial-time quantum algorithm for factoring and discrete log

Grover's algorithm: summary

- **Grover's algorithm:** Quantum algorithm for searching unsorted databases
- **Classical search:** $\mathcal{O}(2^n)$ time to search 2^n items
- **Quantum search:** $\mathcal{O}(2^{n/2})$ time with Grover's algorithm
- **Formal claim:** Given $F : \{0, 1\}^n \rightarrow \{0, 1\}$, Grover's algorithm makes $\mathcal{O}(2^{n/2})$ quantum evaluations of F and outputs $X \in \{0, 1\}^n$ such that $F(X) = 1$, if such X exists
- **Intuition:** Finding needles in haystacks
 - Haystack: $\{0, 1\}^n$ (all possible inputs)
 - Needle: Special input X where $F(X) = 1$

Grover's algorithm: attacking symmetric cryptography

- **Attack setup:** Obtain known plaintext-ciphertext pairs
 - $C_1 = \text{Enc}(K, M_1), C_2 = \text{Enc}(K, M_2), \dots, C_m = \text{Enc}(K, M_m)$
- **Define search function:**

$$F(K) = \begin{cases} 1 & \text{if } \text{Dec}(K, C_i) = M_i \text{ for all } i \\ 0 & \text{otherwise} \end{cases}$$

- **Result:** Grover finds the correct key in $\mathcal{O}(2^{\lambda/2})$ time^a
- **Defense:** Double the key size^b
 - AES-128 $\rightarrow$ AES-256 (to maintain 128-bit security)
 - SHA-256 $\rightarrow$ SHA-512 (for collision resistance)

^aNote: Grover's algorithm is inherently sequential (cannot be parallelized efficiently)

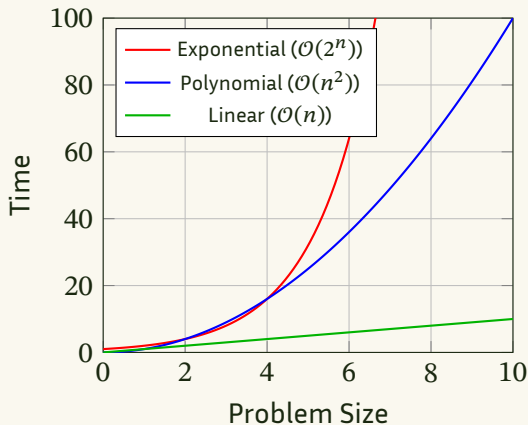
^bCryptographers argue about this recommendation on various points, some thinking it's too simplistic, others thinking that with hash functions, you may not need to go from SHA-256 to SHA-512 with SHA-384 being enough, etc.

Shor's algorithm: summary

- In 1994, Peter Shor demonstrated that quantum computers could break most asymmetric-key cryptography in polynomial time
- **Claim:** There are quantum algorithms for the following tasks, whose running time is polynomial in k :
 - Factoring k -bit numbers
 - Solving the discrete logarithm problem in any cyclic group of order $< 2^k$, even groups based on elliptic curves
- Both algorithms use a common approach called *Shor's algorithm*
- **Implication:** All public-key cryptography based on factoring (RSA) or discrete logarithm (Diffie-Hellman, ElGamal, Schnorr) can be broken by quantum computers

Shor's algorithm: impact

- Classical factoring: Best known algorithm takes exponential time
- Quantum factoring with Shor's: Takes polynomial time
- Example: Factoring a 2048-bit RSA key
 - Classical: Billions of years
 - Quantum: Hours to days (with a large enough quantum computer)
- **Impact:** Complete break of RSA, ECC, and DH key exchange



Not all cryptography is equally vulnerable

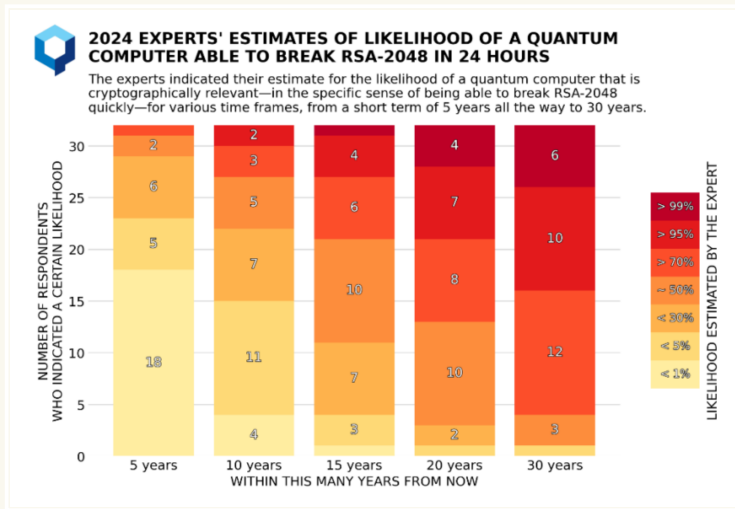
- **Asymmetric cryptography:** Severely impacted by quantum computers
 - RSA, ECC, DH → Completely broken by Shor's algorithm
- **Symmetric cryptography:** Less impacted
 - AES, SHA-256 → Weakened but not broken
 - Grover's algorithm: Provides quadratic speedup
 - Solution: Double the key size (AES-128 → AES-256)^a
- **Hash functions:** Similar to symmetric crypto
 - Need larger output sizes to maintain security

^aCryptographers argue about this recommendation on various points, some thinking it's too simplistic, others thinking that with hash functions, you may not need to go from SHA-256 to SHA-512 with SHA-384 being enough, etc.

Timeline: when will quantum computing arrive?

- **Current state:** Small quantum computers exist (100s of qubits)
- **Required for cryptanalysis:** Millions of error-corrected qubits
- **Expert predictions:** 10-30 years until cryptographically relevant quantum computers (depending on who you ask)
- **But consider:**
 - Technological breakthroughs could accelerate timeline
 - Nation-states investing billions in quantum research
 - Progress has been faster than expected in recent years

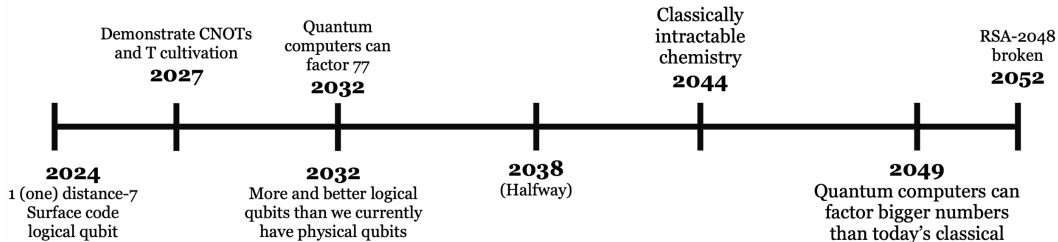
Timeline: when will quantum computing arrive?



Source: Michele Mosca and Marco Piani, Quantum Threat Timeline Report, Global Risk Institute, 2024.

Timeline: according to Sam Jaques

Assume qubits double every 1.5 years and error rate approaches 10^{-3} :



Source: Sam Jaques

Reality check: Current quantum factorization records

- **2001:** IBM factored $15 = 3 \times 5$ using Shor's algorithm
- **2012:** Factored $21 = 3 \times 7$
- **2019:** Attempted to factor $35 = 5 \times 7$
- **In 24 years:** Progress from 15 to 21
- **For context:** A 1981 VIC-20 home computer can factor these numbers instantly
- These aren't even true applications of Shor's algorithm - they use "compiled" versions with prior knowledge of the answer

Alternative “quantum” factorization methods

- Recent paper^a demonstrates matching all quantum factorization records using:
 - **VIC-20**: 8-bit computer from 1981 (538 bytes of code)
 - **Abacus**: Ancient counting device
 - **Dog**: Trained to bark 3 times (factors 15 and 21)
- All methods successfully factor every number claimed by quantum computers
- The VIC-20 can factor the RSA-2048 “quantum” examples in 16.5 seconds
- **Conclusion**: Current quantum computers offer no practical advantage

^a<https://appliedcryptography.page/paper/#quantum-dog>

The sleight-of-hand problem

- Most quantum factorization claims use specially constructed “sleight-of-hand” numbers
- **Example:** RSA-2048 “factorization” claims
 - Factors chosen to differ by only 2 or 6
 - Real RSA keys require factors to differ by >100 bits
 - Can be factored with simple square root calculation
- **Common tricks:**
 - Numbers with special binary patterns (mostly zeros)
 - Preprocessing on classical computers
 - “Compiled” algorithms that know the answer in advance

The gap between hype and reality

- **Claimed:** “RSA-2048 factored by quantum computer”
- **Reality:** Used specially constructed weak keys, solvable on 1980s hardware
- **Estimated timeline:** 10-30 years to cryptographically relevant quantum computers
- **Historical note:** Similar timeline estimates have been made for decades
- **Critical question:** Should we redesign all cryptography based on devices that can't outperform a dog?

But resource estimates are shrinking fast

- **Google Quantum AI (2026)^a:** Compiled quantum circuits for Shor's algorithm targeting 256-bit ECDLP on secp256k1

Variant	Logical Qubits	Toffoli Gates
Low-qubit	$\leq 1,200$	≤ 90 million
Low-gate	$\leq 1,450$	≤ 70 million

- **Physical cost:** $< 500,000$ superconducting qubits (at 10^{-3} error rate)
- **Nearly 20× reduction** over prior estimates for a single ECDLP instance
- The gap between “can't beat a dog” and “could break real crypto” is closing

^a<https://appliedcryptography.page/paper/#cryptocurrency-whitepaper>

Fast-clock vs slow-clock: Why architecture matters

Fast-clock architectures:

- Superconducting, photonic, silicon spin
- Error correction cycle: $\sim 1\ \mu\text{s}$
- ECDLP-256 in **~ 9 minutes**
- Enables real-time *on-spend* attacks

Slow-clock architectures:

- Neutral atom, trapped ion
- 100–1000 $\times$ slower gate times
- ECDLP-256 in **hours to days**
- Only enables *at-rest* attacks on stored data

Which architecture matures first determines the threat model

Store now, decrypt later

- **The problem:** Adversaries can store encrypted data today
- Wait for quantum computers to become available
- Decrypt all previously recorded communications
- **Implications:**
 - Data encrypted today may be decrypted in 10-20 years
 - Long-term secrets are already at risk
 - Medical records, government communications, trade secrets
- **This attack is happening NOW**
- This is why some argue that we can't just not do anything because quantum computers seem impractical *today*

Why transition now?

- **Migration takes time:** Historical crypto transitions took 10-20 years
 - Recall the transition away from RC4, seen earlier in the course!
- **Complex systems:** Need to update protocols, libraries, hardware
- **Backward compatibility:** Must support both old and new systems
- **Unknown vulnerabilities:** New algorithms need time for analysis
- **Compliance and standards:** Regulations take years to update
- **Bottom line:** If we wait until quantum computers exist, it's too late

Section 2

Going Post-Quantum

Post-quantum cryptography: the solution

- **Definition:** Cryptographic algorithms that are secure against both classical and quantum computers
- Based on mathematical problems believed to be hard even for quantum computers:
 - Lattice-based problems
 - Code-based problems
 - Hash-based signatures
 - Multivariate polynomial equations
 - Isogeny-based problems
- **Goal:** Deploy these before quantum computers arrive

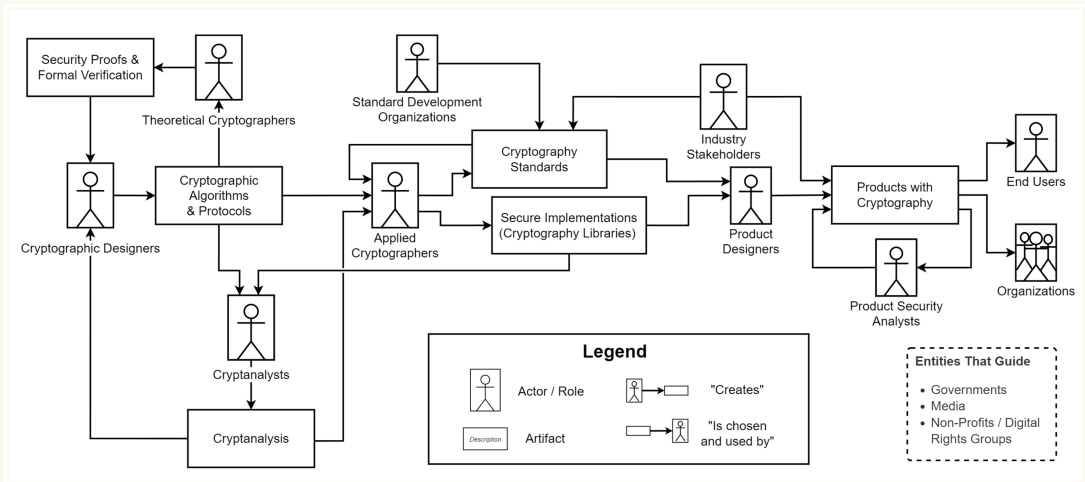
The NIST Post-Quantum Cryptography Competition

- Started in 2016 to standardize post-quantum algorithms
- 69 initial submissions → Multiple rounds of analysis
- **2022**: First algorithms selected for standardization
 - **CRYSTALS-Kyber**: Key encapsulation (KEM)
 - **CRYSTALS-Dilithium**: Digital signatures
 - **FALCON**: Digital signatures
 - **SPHINCS+**: Hash-based signatures
- Global effort: Cryptographers worldwide analyzing security

Real-world impact: who needs to care?

- **Financial institutions:** Protecting transactions and customer data
- **Healthcare:** Medical records must remain private for decades
- **Government:** National security and citizen privacy
- **Technology companies:** Secure communications and data storage
- **Critical infrastructure:** Power grids, water systems, transportation
- **Everyone:** Personal communications, passwords, private data

Starting at the left



Fischer et al, The Challenges of Bringing Cryptography from Research Papers to Products: Results from an Interview Study

Section 2

Going Post-Quantum

Subsection 2.1

Learning with Errors

The Learning with Errors (LWE) Problem

- **Key insight:** Some mathematical problems remain hard even for quantum computers
- **LWE:** Based on solving noisy systems of linear equations
- **Security foundation:** The difficulty of distinguishing random noise from structured noise
- Forms the basis for many post-quantum cryptographic schemes
- First, let's see why regular linear equations are easy...

Solving linear equations: the easy case

- Consider solving for unknowns s_1, s_2, s_3, s_4, s_5 in:

$$3 = s_1 + 2s_2 - 2s_3 + 6s_5$$

$$1 = 4s_1 + 3s_2 - 4s_3 - 3s_4 + 6s_5$$

$$4 = -s_1 - s_2 + 4s_3 + 3s_4 - 3s_5$$

$$1 = s_1 - s_2 + 2s_3 + 3s_4 + s_5$$

$$5 = 4s_1 + 5s_2 + 5s_3 + 3s_4 + s_5$$

- We know an efficient algorithm for this

Matrix form of linear equations

The system can be written as:

$$3 = s_1 + 2s_2 - 2s_3 + 6s_5$$

$$1 = 4s_1 + 3s_2 - 4s_3 - 3s_4 + 6s_5$$

$$4 = -s_1 - s_2 + 4s_3 + 3s_4 - 3s_5$$

$$1 = s_1 - s_2 + 2s_3 + 3s_4 + s_5$$

$$5 = 4s_1 + 5s_2 + 5s_3 + 3s_4 + s_5$$

$$\begin{bmatrix} 3 \\ 1 \\ 4 \\ 1 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 & 2 & -2 & 0 & 6 \\ 4 & 3 & -4 & -3 & 6 \\ -1 & -1 & 4 & 3 & -3 \\ 1 & -1 & 2 & 3 & 1 \\ 4 & 5 & 5 & 3 & 1 \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{bmatrix}$$

- Solution: Invert the matrix and multiply
- Polynomial time algorithm exists
- Adding more constraints (rows) doesn't make it harder

Mathematical note: The equation $\vec{b} = A\vec{s}$ represents a *linear transformation* $T : \mathbb{Z}_q^5 \rightarrow \mathbb{Z}_q^5$ defined by $T(\vec{s}) = A\vec{s}$.

Gaussian elimination $\rightarrow$ *equivalent systems* via row operations that correspond to invertible elementary matrices.

What is Gaussian elimination?

- **Goal:** Solve systems of linear equations systematically
- **Basic idea:** Transform into an equivalent, simpler form
- **Method:** Use row operations to eliminate variables step by step
- **Key insight:** Each row operation creates an *equivalent system*
 - Equivalent $\equiv$ different equations, same solutions
 - Each operation corresponds to multiplication by an elementary matrix
 - Result: Same solution set, easier to solve
- **Notation:**
 - $R_1, R_2, R_3, \dots$ = Row 1, Row 2, Row 3, etc.
- **Augmented matrix** $[A|\vec{b}]$: coefficient matrix with constants attached

Step 1: Set up the augmented matrix

Start with the system:

$$3 = s_1 + 2s_2 - 2s_3 + 6s_5$$

$$1 = 4s_1 + 3s_2 - 4s_3 - 3s_4 + 6s_5$$

$$4 = -s_1 - s_2 + 4s_3 + 3s_4 - 3s_5$$

$$1 = s_1 - s_2 + 2s_3 + 3s_4 + s_5$$

$$5 = 4s_1 + 5s_2 + 5s_3 + 3s_4 + s_5$$

Create augmented matrix $[A|b]$:

$$\left[\begin{array}{ccccc|c} 1 & 2 & -2 & 0 & 6 & 3 \\ 4 & 3 & -4 & -3 & 6 & 1 \\ -1 & -1 & 4 & 3 & -3 & 4 \\ 1 & -1 & 2 & 3 & 1 & 1 \\ 4 & 5 & 5 & 3 & 1 & 5 \end{array} \right]$$

Each row represents one equation

What are row operations?

Three allowed operations (these don't change the solution):

1. Multiply a row by a non-zero constant

- Example: $R_1 \leftarrow 2R_1$ means "multiply Row 1 by 2"

2. Add/subtract a multiple of one row to another

- Example: $R_2 \leftarrow R_2 - 4R_1$ means "subtract 4 times Row 1 from Row 2"

3. Swap two rows

- Example: Swap R_2 and R_3

Our strategy: Use these operations to create zeros below the diagonal

Row operations: Concrete examples

1. Multiply a row by a non-zero constant:

$$\left[\begin{array}{ccc|c} 2 & 4 & 6 & 8 \\ 1 & 3 & 5 & 7 \\ 4 & 2 & 1 & 3 \end{array} \right] \xrightarrow{R_1 \leftarrow \frac{1}{2}R_1} \left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 1 & 3 & 5 & 7 \\ 4 & 2 & 1 & 3 \end{array} \right]$$

2. Add/subtract a multiple of one row to another:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 1 & 3 & 5 & 7 \\ 4 & 2 & 1 & 3 \end{array} \right] \xrightarrow{R_2 \leftarrow R_2 - R_1} \left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 4 & 2 & 1 & 3 \end{array} \right]$$

3. Swap two rows:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 4 & 2 & 1 & 3 \end{array} \right] \xrightarrow{R_1 \leftrightarrow R_3} \left[\begin{array}{ccc|c} 4 & 2 & 1 & 3 \\ 0 & 1 & 2 & 3 \\ 1 & 2 & 3 & 4 \end{array} \right]$$

Step 2a: Understanding the first elimination

Goal: Make all entries in column 1 (below row 1) equal to zero

Current matrix:

$$\left[\begin{array}{ccccc|c} \boxed{1} & 2 & -2 & 0 & 6 & 3 \\ \textcolor{red}{4} & 3 & -4 & -3 & 6 & 1 \\ \textcolor{red}{-1} & -1 & 4 & 3 & -3 & 4 \\ \textcolor{red}{1} & -1 & 2 & 3 & 1 & 1 \\ \textcolor{red}{4} & 5 & 5 & 3 & 1 & 5 \end{array} \right]$$

- The boxed 1 is our **pivot** (the element we use to eliminate others)
- We want to turn the red numbers into zeros

Step 2b: Performing the eliminations

To eliminate the 4 in Row 2: subtract 4 times Row 1 from Row 2

- $R_2 \leftarrow R_2 - 4R_1$ means:
 $(4, 3, -4, -3, 6|1) - 4(1, 2, -2, 0, 6|3)$
- This gives: $(0, -5, 4, -3, -18|-11)$

Similarly:

- $R_3 \leftarrow R_3 + R_1$ (add Row 1 to Row 3 to eliminate the -1)
- $R_4 \leftarrow R_4 - R_1$ (subtract Row 1 from Row 4 to eliminate the 1)
- $R_5 \leftarrow R_5 - 4R_1$ (subtract 4 times Row 1 from Row 5)

Result:

$$\left[\begin{array}{ccccc|c} 1 & 2 & -2 & 0 & 6 & 3 \\ 0 & -5 & 4 & -3 & -18 & -11 \\ 0 & 1 & 2 & 3 & 3 & 7 \\ 0 & -3 & 4 & 3 & -5 & -2 \\ 0 & -3 & 13 & 3 & -23 & -7 \end{array} \right]$$

What is a pivot and why swap rows?

Pivot: The number we use to eliminate entries below it

Looking at column 2:

$$\left[\begin{array}{ccccc|c} 1 & 2 & -2 & 0 & 6 & 3 \\ 0 & -5 & 4 & -3 & -18 & -11 \\ 0 & 1 & 2 & 3 & 3 & 7 \\ 0 & -3 & 4 & 3 & -5 & -2 \\ 0 & -3 & 13 & 3 & -23 & -7 \end{array} \right]$$

- We could use -5 as our pivot, but that requires fractions
- The blue 1 is simpler! (No fractions needed)
- So we swap rows 2 and 3 to make 1 our pivot

Step 3: Second column elimination

After swapping rows 2 and 3:

$$\left[\begin{array}{ccccc|c} 1 & 2 & -2 & 0 & 6 & 3 \\ 0 & \boxed{1} & 2 & 3 & 3 & 7 \\ 0 & -5 & 4 & -3 & -18 & -11 \\ 0 & -3 & 4 & 3 & -5 & -2 \\ 0 & -3 & 13 & 3 & -23 & -7 \end{array} \right]$$

Now eliminate the red entries:

- $R_3 \leftarrow R_3 + 5R_2$ (add 5 times Row 2 to eliminate -5)
- $R_4 \leftarrow R_4 + 3R_2$ (add 3 times Row 2 to eliminate -3)
- $R_5 \leftarrow R_5 + 3R_2$ (add 3 times Row 2 to eliminate -3)

What is row echelon form?

After continuing this process:

$$\left[\begin{array}{ccccc|c} 1 & 2 & -2 & 0 & 6 & 3 \\ 0 & 1 & 2 & 3 & 3 & 7 \\ 0 & 0 & 14 & 12 & -3 & 24 \\ 0 & 0 & 10 & 12 & 4 & 19 \\ 0 & 0 & 19 & 12 & -14 & 14 \end{array} \right]$$

Row echelon form has:

- Leading entry (first non-zero) in each row is to the right of the one above
- All zeros below each leading entry
- Forms a “staircase” pattern

We continue until we get this form for all columns

Back substitution: solving from the bottom up

Once in row echelon form, we can solve:

- **Last equation:** Gives us one variable directly
- **Second-to-last:** Substitute known value, solve for next variable
- **Continue upward:** Each equation gives us one more variable

Example (simplified):

$$x + 2y + 3z = 10$$

$$y + 2z = 5$$

$$z = 1$$

From bottom: $z = 1$, then $y = 5 - 2(1) = 3$, then $x = 10 - 2(3) - 3(1) = 1$

Why is this process efficient?

Gaussian elimination achieves polynomial-time complexity:

- **Theoretical complexity:** $O(n^3)$ for $n \times n$ system
 - Compare to brute force: $O(q^n)$ where q is field size
 - For $n = 1000$: $\sim 10^9$ operations vs q^{1000} (astronomical!)
- **Algorithmic refinements** for practical speed:
 - **Pivoting strategies:** Choose best pivot to minimize numerical errors
 - **Sparse methods:** Exploit matrices with mostly zeros
 - **Block algorithms:** Optimize for memory cache hierarchies
- **Hardware acceleration** (critical for modern implementations):
 - **GPU parallelism:** Thousands of cores compute simultaneously
 - **SIMD instructions:** Process multiple data elements per CPU cycle
 - **Optimized libraries:** BLAS, LAPACK, cuBLAS exploit hardware
 - **Real speedup:** 50–100× faster with good implementation
- **Works in finite fields:** Same techniques apply modulo q

Bottom line: Modern systems solve $n \sim 10^6$ variables in seconds!

Making it hard: approximate equations

Now consider *approximate* equations:

$$2 \approx s_1 + 2s_2 - 2s_3 + 6s_5$$

$$7 \approx 4s_1 + 3s_2 - 4s_3 - 3s_4 + 6s_5$$

$$1 \approx -s_1 - s_2 + 4s_3 + 3s_4 - 3s_5$$

$$8 \approx s_1 - s_2 + 2s_3 + 3s_4 + s_5$$

$$2 \approx 4s_1 + 5s_2 + 5s_3 + 3s_4 + s_5$$

$$8 \approx 2s_1 + s_3 - s_4 + 2s_5$$

...where $\approx$ means “equal up to small error” (e.g., ± 1). **This breaks Gaussian elimination** by introducing small errors that destroy the exact cancellations we rely on!

The Learning with Errors formulation

We can rewrite approximate equations exactly by adding error terms:

$$\begin{bmatrix} 2 \\ 7 \\ 1 \\ 8 \\ 2 \\ 8 \\ \vdots \end{bmatrix} = \begin{bmatrix} 1 & 2 & -2 & 0 & 6 \\ 4 & 3 & -4 & -3 & 6 \\ -1 & -1 & 4 & 3 & -3 \\ 1 & -1 & 2 & 3 & 1 \\ 4 & 5 & 5 & 3 & 1 \\ 2 & 0 & 1 & -1 & 2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ \vdots \end{bmatrix}$$

where error vector $\vec{e} = (e_1, e_2, e_3, \dots)$ has small entries from $\{-1, 0, +1\}$

Why LWE is hard

- **The dichotomy:**
 - With many equations (tall matrix), there's usually at most one solution
 - But finding that solution appears to be extremely hard!
- **All known algorithms require exponential time**
 - This includes quantum algorithms
 - No **known** quantum speedup like Shor's algorithm
- **Intuition:** Small errors destroy the algebraic structure
 - Can't simply invert the matrix anymore
 - Errors interact in complex ways
 - Finding the secret requires distinguishing structure from noise

LWE for cryptography

- **Public key:** Matrix A and noisy products $b = As + e$
- **Secret key:** The secret vector s
- **Security:** Based on hardness of recovering s from (A, b)
- **Parameters:**
 - Dimension n : Size of secret vector
 - Modulus q : Work in $\mathbb{Z}_q$
 - Error distribution: Typically Gaussian with small standard deviation
- **Applications:** Key exchange (Kyber), signatures (Dilithium), fully homomorphic encryption

Learning with Errors (LWE): The core idea

- **Recall:** We saw that approximate linear equations are hard to solve
- **LWE:** A cryptographic problem based on this hardness
- **Setup:** Given matrix M and vector $\vec{x} = M\vec{s} + \vec{e}$
 - $\vec{s}$: Secret vector (sampled uniformly)
 - $\vec{e}$: Error/noise vector (sampled from distribution of “small” values)
 - All arithmetic performed modulo q
- **The challenge:** Distinguish LWE samples from random vectors

The LWE distinguishing challenge: What does it mean?

Scenario 1: LWE samples

- Alice picks secret $\vec{s}$
- Computes $\vec{x} = M\vec{s} + \vec{e}$
- Gives you $(M, \vec{x})$

Scenario 2: Random

- Alice picks random $\vec{x}$
- Gives you $(M, \vec{x})$

Your task: Tell which scenario you're in!

Why is this hard?

- Noise $\vec{e}$ makes $\vec{x}$ look random
- Without the secret $\vec{s}$, can't tell the difference
- Like trying to find a hidden pattern in static

Cryptographic use:

- If you can't distinguish, you can't break the crypto
- Real samples $\rightarrow$ encrypted messages
- Random samples $\rightarrow$ meaningless noise
- Adversary can't tell which is which!

LWE: Mathematical formulation

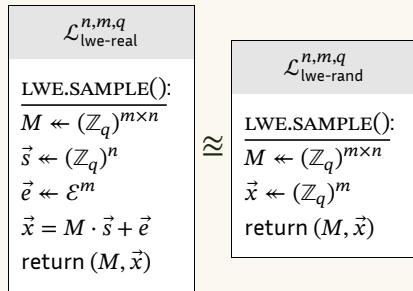
LWE sample generation:

1. Sample matrix $M \in \mathbb{Z}_q^{m \times n}$
2. Sample secret $\vec{s} \in \mathbb{Z}_q^n$
3. Sample noise $\vec{e} \in \mathbb{Z}_q^m$ (small)
4. Compute: $\vec{x} = M\vec{s} + \vec{e}$
5. Output: $(M, \vec{x})$

Visual representation:

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}}_{\vec{x}} = \underbrace{\begin{bmatrix} - & \text{row 1} & - \\ - & \text{row 2} & - \\ & \vdots & \\ - & \text{row m} & - \end{bmatrix}}_M \underbrace{\begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}}_{\vec{s}} + \underbrace{\begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{bmatrix}}_{\vec{e}}$$

The LWE hardness assumption



- **The LWE assumption:** These two distributions are computationally indistinguishable **even for quantum computers**: no known quantum speedup.

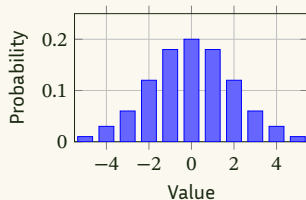
Understanding “small” in modular arithmetic

- **Working modulo q :** Think of $\mathbb{Z}_q$ as a circle
- **“small” means “close to 0”** — in either direction!
 - Example: If $q = 11$, then $10 \equiv -1 \pmod{11}$ is “small”
- **Centered representation:** Instead of $\{0, 1, \dots, q-1\}$, use:

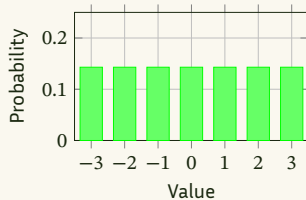
$$\left\{ -\left\lfloor \frac{q-1}{2} \right\rfloor, \dots, -1, 0, 1, \dots, \left\lceil \frac{q-1}{2} \right\rceil \right\}$$

- **Common noise distributions:**
 - Discrete Gaussian (bell curve centered at 0)
 - Uniform over $\{-c, -c+1, \dots, c-1, c\}$ for small c

Discrete Gaussian



Uniform



Key Agreement vs Key Encapsulation

Shared Secret Agreement (DH-style):

- Both parties contribute randomness
- Alice: g^a , Bob: g^b
- Shared secret: g^{ab}
- **Interactive:** Both parties actively compute
- Traditional approach (DH, ECDH)

Key Encapsulation (KEM):

- One party generates the key
- Alice: Generates random key k
- Alice: Encapsulates k in ciphertext c
- Bob: Decapsulates key k from ciphertext c
- **Asymmetric:** One generates, one recovers

Both achieve the same goal: Establish a shared secret key

From LWE to key encapsulation

- **Key insight:** LWE naturally supports key encapsulation mechanisms (KEMs)
- **KEM approach:** One party generates and encapsulates a key
- **Recipient:** Uses private key to decapsulate and recover the key
- **Why KEMs?:**
 - Lattice problems don't naturally support DH-style operations
 - No efficient "commutative" operation like $g^{ab} = g^{ba}$
 - KEMs are the natural construction for lattice-based crypto
- **Reconciliation:** Still needed to extract identical keys from noisy values

Mathematical notation primer

- **Vectors:** Elements of $\mathbb{Z}_q^n$, written $\vec{v} \in \mathbb{Z}_q^n$

- By abuse of notation, we represent $\vec{v}$ as an $n \times 1$ matrix (column): $\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$

- This convention enables matrix-vector multiplication: $A\vec{v}$
- Vectors aren't inherently "row" or "column"—that's a matrix representation

- **Matrices:** 2D arrays, e.g., $A \in \mathbb{Z}_q^{n \times n}$ (n rows, n columns)

- **Matrix transpose** A^T : Flip rows and columns, so $(A^T)_{ij} = A_{ji}$

- For vectors: $\vec{v}^T$ transposes the $n \times 1$ column to $1 \times n$ row

- Example: $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^T = [1 \quad 2 \quad 3]$

- $\vec{v}^T \vec{w}$ is valid ($1 \times n$ times $n \times 1 = \text{scalar}$) but $\vec{v} \vec{w}$ is undefined ($n \times 1$ times $n \times 1$)

Sampling and distribution notation

Sampling notation:

- $x \leftarrow S$ means “randomly pick x from set S ”
- Like rolling dice or drawing from a hat

Error distribution $\mathcal{E}$:

- A probability distribution over small integers

Sampling examples:

- $e \leftarrow \mathcal{E}$
 - Sample one small integer
 - Result: e.g., $e = -1$
- $\vec{e} \leftarrow [\mathcal{E}]^n$
 - Sample n integers independently
 - Result: e.g., $\vec{e} = \begin{bmatrix} 0 \\ 1 \\ -1 \\ 0 \end{bmatrix}$

Other notation in the KEM

- **Floor function:** $\lfloor x \rfloor$ means round down to nearest integer
 - $\lfloor 3.7 \rfloor = 3, \lfloor 5.1 \rfloor = 5$
 - $\lfloor q/2 \rfloor$ = halfway point in $\mathbb{Z}_q$
 - If $q = 3329$, then $\lfloor q/2 \rfloor = 1664$
- **Matrix-vector multiplication (dimensions matter!):**
 - $A\vec{s}$ where $A \in \mathbb{Z}_q^{m \times n}$ and $\vec{s} \in \mathbb{Z}_q^n$: $(A\vec{s})_i = \sum_{j=1}^n A_{ij}s_j$ (result is vector in $\mathbb{Z}_q^m$)
 - $\vec{b}^T \vec{r}$ where $\vec{b}, \vec{r} \in \mathbb{Z}_q^n$ (inner product): $\vec{b}^T \vec{r} = \sum_{i=1}^n b_i r_i$ (result is scalar in $\mathbb{Z}_q$)
 - The transpose T ensures dimensional consistency: $1 \times n$ times $n \times 1$ yields 1×1 (scalar)
- **The shared bit μ :**
 - The actual message being transmitted: 0 or 1
 - Encoded by adding 0 (if $\mu = 0$) or $\lfloor q/2 \rfloor$ (if $\mu = 1$)
 - Small errors won't confuse 0 with $\lfloor q/2 \rfloor$

Simple LWE-based KEM

Public parameters:

- Matrix $A \in \mathbb{Z}_q^{n \times n}$
- Modulus q (e.g., 3329)^a
- Error distribution $\mathcal{E}$

Bob's key generation:

1. Sample $\vec{s}, (\vec{e}_1 \leftarrow [\mathcal{E}]^n)$
2. Private key: $\vec{s}$
3. Public key: $\vec{b} = A\vec{s} + \vec{e}_1$
4. Publish $(A, \vec{b})$

^a3329 is congruent to 1 modulo 256, and therefore the polynomial $X^{256} + 1$ splits into a product of degree 2 polynomials of the form $(X^2 - r_i)$.

^b<https://appliedcryptography.page/paper/#basic-lattice>

Alice's encapsulation:

1. Sample $\vec{r}, (\vec{e}_2 \leftarrow [\mathcal{E}]^n)$
2. Sample $e_3 \leftarrow \mathcal{E}$
3. Message bit: $\mu \in \{0, 1\}$
4. Compute:
 - $\vec{u}^T = \vec{r}^T A + \vec{e}_2^T$
 - $v = \vec{r}^T \vec{b} + e_3 + \lfloor q/2 \rfloor \cdot \mu$
5. Send ciphertext: $(\vec{u}, v)$

Bob's decapsulation:

1. Compute: $w = v - \vec{u}^T \vec{s}$
2. If w closer to 0: $\mu = 0$
3. If w closer to $\lfloor q/2 \rfloor$: $\mu = 1$

Why decapsulation works

$$\begin{aligned}v - \vec{u}^T \vec{s} &= \vec{r}^T (A\vec{s} + \vec{e}_1) + e_3 + \lfloor q/2 \rfloor \cdot \mu - (\vec{r}^T A + \vec{e}_2^T) \vec{s} \\&= \vec{r}^T A\vec{s} + \vec{r}^T \vec{e}_1 + e_3 + \lfloor q/2 \rfloor \cdot \mu - \vec{r}^T A\vec{s} - \vec{e}_2^T \vec{s} \\&= \vec{r}^T \vec{e}_1 + e_3 + \lfloor q/2 \rfloor \cdot \mu - \vec{e}_2^T \vec{s} \\&= \lfloor q/2 \rfloor \cdot \mu + \underbrace{(\vec{r}^T \vec{e}_1 - \vec{e}_2^T \vec{s} + e_3)}_{\text{small error}}\end{aligned}$$

Why decapsulation works

$$\begin{aligned}
 v - \vec{u}^T \vec{s} &= \vec{r}^T (A\vec{s} + \vec{e}_1) + e_3 + \lfloor q/2 \rfloor \cdot \mu - (\vec{r}^T A + \vec{e}_2^T) \vec{s} \\
 &= \vec{r}^T A\vec{s} + \vec{r}^T \vec{e}_1 + e_3 + \lfloor q/2 \rfloor \cdot \mu - \vec{r}^T A\vec{s} - \vec{e}_2^T \vec{s} \\
 &= \vec{r}^T \vec{e}_1 + e_3 + \lfloor q/2 \rfloor \cdot \mu - \vec{e}_2^T \vec{s} \\
 &= \lfloor q/2 \rfloor \cdot \mu + \underbrace{(\vec{r}^T \vec{e}_1 - \vec{e}_2^T \vec{s} + e_3)}_{\text{small error}}
 \end{aligned}$$

- **Step 1:** Start with $v - \vec{u}^T \vec{s}$ and substitute the definitions
 - Recall: $v = \vec{r}^T \vec{b} + e_3 + \lfloor q/2 \rfloor \cdot \mu$ where $\vec{b} = A\vec{s} + \vec{e}_1$
 - So: $v = \vec{r}^T (A\vec{s} + \vec{e}_1) + e_3 + \lfloor q/2 \rfloor \cdot \mu$
 - And: $\vec{u}^T = \vec{r}^T A + \vec{e}_2^T$
 - Substituting:

$$v - \vec{u}^T \vec{s} = \vec{r}^T (A\vec{s} + \vec{e}_1) + e_3 + \lfloor q/2 \rfloor \cdot \mu - (\vec{r}^T A + \vec{e}_2^T) \vec{s}$$
- **Step 2:** Expand the products using distributivity
 - $\vec{r}^T (A\vec{s} + \vec{e}_1) = \vec{r}^T A\vec{s} + \vec{r}^T \vec{e}_1$
 - $(\vec{r}^T A + \vec{e}_2^T) \vec{s} = \vec{r}^T A\vec{s} + \vec{e}_2^T \vec{s}$
 - Result:

$$\vec{r}^T A\vec{s} + \vec{r}^T \vec{e}_1 + e_3 + \lfloor q/2 \rfloor \cdot \mu - \vec{r}^T A\vec{s} - \vec{e}_2^T \vec{s}$$

Why decapsulation works

$$\begin{aligned}v - \vec{u}^T \vec{s} &= \vec{r}^T (A\vec{s} + \vec{e}_1) + e_3 + [q/2] \cdot \mu - (\vec{r}^T A + \vec{e}_2^T) \vec{s} \\&= \vec{r}^T A\vec{s} + \vec{r}^T \vec{e}_1 + e_3 + [q/2] \cdot \mu - \vec{r}^T A\vec{s} - \vec{e}_2^T \vec{s} \\&= \vec{r}^T \vec{e}_1 + e_3 + [q/2] \cdot \mu - \vec{e}_2^T \vec{s} \\&= [q/2] \cdot \mu + \underbrace{(\vec{r}^T \vec{e}_1 - \vec{e}_2^T \vec{s} + e_3)}_{\text{small error}}\end{aligned}$$

- **Step 3:** Cancel the $\vec{r}^T A\vec{s}$ terms
 - Notice: $+\vec{r}^T A\vec{s}$ and $-\vec{r}^T A\vec{s}$ cancel out
 - Left with: $\vec{r}^T \vec{e}_1 + e_3 + [q/2] \cdot \mu - \vec{e}_2^T \vec{s}$
- **Step 4:** Rearrange to isolate the message
 - Group error terms: $\vec{r}^T \vec{e}_1 - \vec{e}_2^T \vec{s} + e_3$
 - Final form: $[q/2] \cdot \mu + (\vec{r}^T \vec{e}_1 - \vec{e}_2^T \vec{s} + e_3)$
- **Why this works:**
 - All error terms ($\vec{e}_1, \vec{e}_2, e_3, \vec{r}, \vec{s}$) are small
 - Their products and sums remain small
 - If $\mu = 0$: Result is close to 0
 - If $\mu = 1$: Result is close to $[q/2]$
 - Small errors don't confuse these two cases!

Reconciliation: Getting exact agreement

- **Problem:** Noise might cause decapsulation errors
- **Solution:** Error correction codes or reconciliation mechanisms
- **Modern approach:**
 1. Encode message bits with error correction
 2. Add reconciliation data to help correct errors
 3. Ensure Bob always recovers Alice's intended key
- **Example (simplified):**
 - Instead of single bit μ , encode multiple bits
 - Use redundancy to correct small errors
 - Kyber uses sophisticated reconciliation for reliability

From theory to practice: Kyber

- **CRYSTALS-Kyber:** NIST-standardized LWE-based KEM
- **Key improvements over basic scheme:**
 - Uses structured matrices (polynomial rings) for efficiency
 - Module-LWE instead of plain LWE
 - Careful parameter selection for security/performance balance
 - CCA-secure construction using Fujisaki-Okamoto transform
- **Performance:** Competitive with current elliptic curve cryptography
- **Already deployed:** Chrome, Cloudflare, AWS, and others

The Fujisaki-Okamoto Transform

Reminder

- Many public-key schemes (RSA, ElGamal) are only CPA-secure.
- The **Fujisaki-Okamoto (FO) transform** converts CPA-secure PKE to CCA-secure PKE.
- Basic idea:
 - Use randomness derived from the message itself.
 - Add redundancy that can be verified during decryption.
 - Invalid ciphertexts will fail verification.

FO Transform (simplified)

Given CPA-secure PKE scheme Π :

- $\text{Enc}_{FO}(pk, M)$:
 1. $R := H(M)$
 2. $C_1 := \Pi.\text{Enc}(pk, M; R)$
 3. $C_2 := G(M) \oplus R'$
 4. Return (C_1, C_2)
- Decryption verifies consistency

LWE/Lattice-based crypto: The trade-offs

Advantages:

- ✓ Fast operations
- ✓ Both encryption & signatures
- ✓ NIST standardized
- ✓ Efficient implementations
- ✓ Flexible parameters

Disadvantages:

- ✗ Newer assumptions (20 years)
- ✗ Larger than classical crypto
- ✗ Complex parameter selection
- ✗ Implementation pitfalls

Bottom line: Leading choice for general-purpose post-quantum cryptography

Section 2

Going Post-Quantum

Subsection 2.2

Code-Based Cryptography

Another PQ candidate: Code-based cryptography

- **Different foundation:** Based on error-correcting codes
- **Long history:** Theory dates back to 1950s
- **Battle-tested:** McEliece cryptosystem (1978) still unbroken after 45+ years!
- **The trade-off:** Very secure, but public keys are large (100+ KB)
- **Modern perspective:** Is 100 KB really too big when average webpage is 2 MB?

What are error-correcting codes?

- **The problem:** Transmitting data over noisy channels
 - You send: 010
 - Channel flips a bit (hardware fault, solar flare...)
 - Receiver gets: 011 (wrong!)
- **The solution:** Add redundancy to detect and correct errors
- **Real-world applications:**
 - CDs and DVDs (scratches don't destroy data)
 - Deep space communication (signals from Mars rovers)
 - QR codes (work even when partially damaged)
 - Computer memory (cosmic rays can flip bits!)



Thanks to error correction, this QR code will still work even if you cover *any* small part of it with your hand. Try it!

The simplest code: Repetition

Basic idea: Repeat each bit multiple times

- Original: 010
- Encoded: 000 111 000
- Each bit repeated 3 times

Decoding: Take majority vote

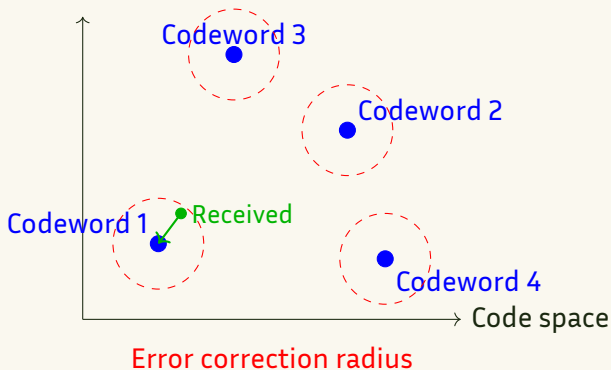
- Received: 100 110 111
- Decode: 0 (two 0s) 1 (two 1s) 1 (three 1s)
- Result: 011

Limitation: Can only correct 1 error per 3-bit chunk

Linear codes: More sophisticated error correction

- **Key idea:** Use linear algebra instead of simple repetition
- **Encoding process:**
 1. Start with n -bit message vector $\vec{v}$
 2. Multiply by generator matrix G (size $n \times m$, where $m > n$)
 3. Get codeword: $\vec{w} = \vec{v} \cdot G$
- **The magic:** Matrix G is designed so that:
 - Codewords are spread far apart in “code space”
 - Can correct up to t errors (designer’s choice)
 - More errors $\rightarrow$ larger codewords needed
- **Efficiency:** Much better than repetition codes!

Visualizing linear codes



- Codewords are far apart
- Errors move you within the “sphere”
- Decode to nearest codeword

The McEliece cryptosystem (1978)

- **Revolutionary idea:** Use error correction for encryption!
- **Public key:** Generator matrix $G = ABC$ (looks random)
 - A, B, C are special matrices (kept secret)
 - Product looks like random matrix
- **Encryption:**
 1. Encode message: $\vec{w} = \vec{m} \cdot G$
 2. Add controlled errors: $\vec{c} = \vec{w} + \vec{e}$
 3. $\vec{e}$ has exactly t bits set to 1
- **Decryption:** Only possible if you know A, B, C
 - Use secret structure to remove errors
 - Recover original message

McEliece encryption: The basic setup

Bob creates his keys:

1. Pick a “nice” error-correcting code
 - Has special structure (secret)
 - Can efficiently correct t errors
 - Generator matrix: G'
2. Disguise it with random matrices
 - S : Random invertible matrix
 - P : Random permutation matrix
 - Compute: $G = S \cdot G' \cdot P$
3. Public key: G (looks random!)
4. Private key: (S, G', P)

McEliece encryption: How Alice sends a message

Alice wants to send message m to Bob:

1. Encode using Bob's public key (G, t)
 - Compute: $c_0 = m \cdot G$
 - This is a valid codeword
2. Add random errors
 - Pick error vector e
 - Exactly t bits set to 1
 - Compute: $c = c_0 + e$
3. Send ciphertext c to Bob

Why add errors?

- Makes c look random
- Hides the structure
- Like adding noise to a signal

McEliece decryption: How Bob recovers the message

Bob receives $c = c_0 + e$ from Alice:

1. Undo the permutation using **his private key P**
 - Compute: $c' = c \cdot P^{-1}$
 - This gives: $c' = m \cdot S \cdot G' + e'$
2. Use special structure of **his private key G'**
 - Error-correct c' using G'
 - Removes the t errors
 - Gets: $m \cdot S \cdot G'$
3. Recover original message using **his private key S**
 - Apply S^{-1} to get m

Why is McEliece secure?

- **The underlying problem:** Decoding random linear codes
 - Given: Random-looking matrix and noisy codeword
 - Find: Original message
 - This is NP-hard! (No efficient algorithm known)
- **Quantum resistance:**
 - No quantum algorithm gives significant speedup
 - Unlike factoring (Shor's algorithm)
 - 45+ years of cryptanalysis, still secure
- **Important caveat:** Not all instances of NP-hard problems are hard
 - Need careful parameter selection
 - Decades of research have refined secure choices

Code-based crypto: Pros and cons

Advantages:

- ✓ Long history (45+ years)
- ✓ Well-understood security
- ✓ Fast operations
- ✓ Simple to implement
- ✓ Quantum resistant

Disadvantages:

- ✗ Large public keys (100+ KB)
- ✗ Not selected by NIST (yet?)
- ✗ Less deployment experience

Perspective: Is 100 KB really too big today?

Section 2

Going Post-Quantum

Subsection 2.3

Hash-Based Cryptography

Hash-based cryptography: A different approach

- **Fundamental difference:** Not based on hard mathematical problems!
- **Instead:** Based on security of cryptographic hash functions
- **Key insight:** Quantum computers don't break hash functions
 - No known quantum algorithm that breaks collision resistance
 - No quantum speedup for finding preimages (beyond Grover's)
- **Security foundation:** If you can't invert hashes, you can't forge signatures
- **History:** Ideas date back to 1979 (Lamport, Winternitz)

The building block: One-time signatures

- **One-time signature:** Can sign only ONE message with each key
- **Why accept this limitation?**
 - Extremely simple construction
 - Provably secure (if hash is secure)
 - Building block for multi-use schemes
- **Real-world analogy:** Like a wax seal
 - Break the seal to verify authenticity
 - Can't reuse it for another letter
 - Need a new seal for each message



Letter sealed with red wax.

Winternitz One-Time Signature (WOTS)

Setup (parameter w = maximum value):

- Private key: Random string K
- Public key: $\text{Hash}^w(K)$
 - Apply hash w times to get the public key

To sign message M (where $0 \leq M \leq w - 1$):

- First hash the actual message to get a value in range $[0, w - 1]$
- Signature: $\text{Hash}^M(K)$
- Apply hash M times to K

Example with $w = 256$:

- Can sign any byte (0-255)
- Hash message $\rightarrow$ byte value M
- Sign by computing $\text{Hash}^M(K)$

WOTS verification: Why it works

- **Given:** Signature $S = \text{Hash}^M(K)$ for message M
- **Verification:** Check if $\text{Hash}^{w-M}(S) = \text{Public Key}$
- **Why this works:**

$$\begin{aligned}\text{Hash}^{w-M}(S) &= \text{Hash}^{w-M}(\text{Hash}^M(K)) \\ &= \text{Hash}^{(w-M)+M}(K) \\ &= \text{Hash}^w(K) \\ &= \text{Public Key} \checkmark\end{aligned}$$

- **Security:** Can't go backwards in the hash chain!
 - Given $\text{Hash}^5(K)$, can't compute $\text{Hash}^3(K)$
 - Would require inverting the hash function

WOTS limitation 1: Signature forgery

- **Critical flaw:** Can forge signatures for larger messages!
- **Example:**
 - You sign message $M = 3$: $S = \text{Hash}^3(K)$
 - Attacker computes: $\text{Hash}(S) = \text{Hash}^4(K)$
 - This is a valid signature for message $M = 4$!
- **General attack:** From signature of M , can forge any $M' > M$
- **Fix:** Sign both M and $w - M$ using two keys
 - Signature: $(\text{Hash}^M(K_1), \text{Hash}^{w-M}(K_2))$
 - Now increasing M requires decreasing $w - M$
 - But you can't go backwards in hash chains!

WOTS limitation 2: Message length

The problem with long messages:

- 8-bit message: $M \in [0, 255]$
 - Need up to 255 hash operations
 - Manageable
- 32-bit message: $M \in [0, 2^{32} - 1]$
 - Need up to 4 billion hashes!
 - Impractical
- 128-bit message: $M \in [0, 2^{128} - 1]$
 - Need up to 2^{128} hashes
 - Impossible

Solution: Split into chunks

- Break 32-bit message into four 8-bit chunks
- Sign each chunk independently
- Need 4 key pairs instead of 1
- Maximum 255 hashes per chunk

Trade-off:

- More keys = larger signatures
- But computation stays feasible

WOTS limitation 3: The one-time problem

- **Critical security requirement:** Never reuse a key!
- **What happens if you sign two messages?**
 - Sign $M_1 = 2$: Reveal $\text{Hash}^2(K)$
 - Sign $M_2 = 7$: Reveal $\text{Hash}^7(K)$
 - Attacker can now compute $\text{Hash}^x(K)$ for any $x \in [2, 7]$
 - Can forge signatures for messages 3, 4, 5, 6!
- **With the two-key fix:** Even worse
 - Multiple signatures reveal different points on both chains
 - Attacker gains more forgery capabilities
- **No simple fix:** This is inherent to the construction

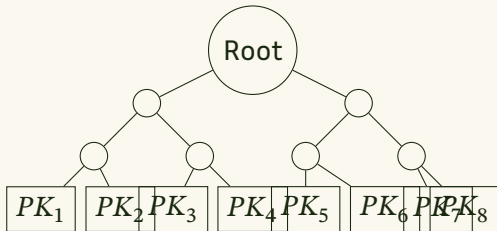
From one-time to many-time: Merkle trees

The idea: Use many OTS keys

- Generate 2^h one-time key pairs
- Build a Merkle tree of public keys
- Tree root = master public key
- Each leaf = one signature

To sign:

- Use next unused OTS key
- Include path to root as proof
- Mark key as used (stateful!)



Modern hash-based signatures

- **XMSS (eXtended Merkle Signature Scheme):**
 - Stateful: Must track which keys are used
 - Smaller signatures than SPHINCS+
 - Risk: Reusing state breaks security
- **SPHINCS+ (NIST standard: SLH-DSA):**
 - Stateless: No need to track used keys
 - Larger signatures (tens of KB)
 - Uses sophisticated tree structures
 - “Few-time” signatures instead of one-time
- **Key limitation:** Signatures only!
 - Hash functions are one-way
 - Can’t build encryption from one-way functions alone

Hash-based crypto: The trade-offs

Advantages:

- ✓ Simplest security assumption
- ✓ Well-understood (40+ years)
- ✓ Quantum resistant
- ✓ Fast verification
- ✓ Can be stateless

Disadvantages:

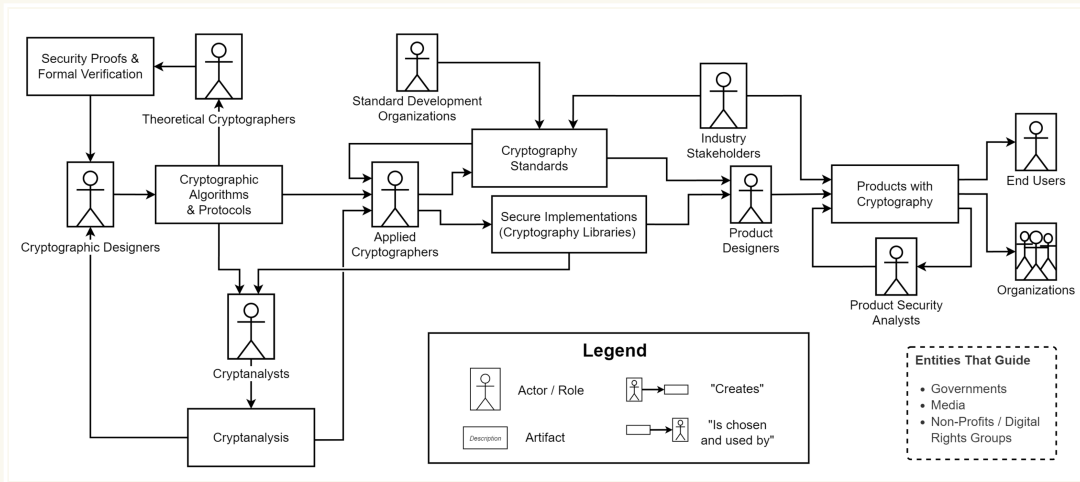
- ✗ Large signatures (10-50 KB)
- ✗ Signatures only (no encryption)
- ✗ Complex/costly key generation
- ✗ Stateful variants risky

Bottom line: Most conservative choice for long-term signatures

Section 3

Today's Post-Quantum Protocols

Moving towards the middle



Fischer et al, The Challenges of Bringing Cryptography from Research Papers to Products: Results from an Interview Study

NIST PQ standardization: The winners

- **August 2024:** NIST released final standards after 8-year process
- **Three primary standards** (with new names):

Original Name	Standard Name	Hard Problem
<i>Key Encapsulation</i>		
CRYSTALS-Kyber	ML-KEM	Module LWE
<i>Digital Signatures</i>		
CRYSTALS-Dilithium	ML-DSA	Module LWE
FALCON	FN-DSA	NTRU lattice
SPHINCS+	SLH-DSA	Hash functions

- **ML:** Module Lattice
- **FN:** Fast Fourier transform over NTRU
- **SLH:** Stateless Hash-based

Key Agreement vs Key Encapsulation

Shared Secret Agreement (DH-style):

- Both parties contribute randomness
- Alice: g^a , Bob: g^b
- Shared secret: g^{ab}
- **Interactive:** Both parties actively compute
- Traditional approach (DH, ECDH)

Key Encapsulation (KEM):

- One party generates the key
- Alice: Generates random key k
- Alice: Encapsulates $k \rightarrow$ ciphertext c
- Bob: Decapsulates $c \rightarrow$ key k
- **Asymmetric:** One generates, one recovers

Both achieve the same goal: Establish a shared secret key

KEMs use public/private key pairs

- **KEMs still use asymmetric cryptography:**
 - Bob has a public/private key pair (pk_B, sk_B)
 - Bob's public key is shared openly
 - Bob's private key is kept secret
- **The KEM process:**
 1. Alice obtains Bob's public key pk_B
 2. Alice generates random shared secret k
 3. Alice encapsulates: $c = \text{Encaps}(pk_B, k)$
 4. Alice sends ciphertext c to Bob
 5. Bob decapsulates: $k = \text{Decaps}(sk_B, c)$
- **Security guarantee:** Only Bob (with sk_B) can recover k from c

Why KEMs for Post-Quantum?

Why not do shared secret agreement like Diffie-Hellman?

- **Efficiency with lattices/codes:**
 - Natural fit: “Encrypt” a random key with public key
 - Avoid complex reconciliation mechanisms
 - Better bandwidth usage
 - Potentially fewer round trips
- **Mathematical limitations:**
 - Lattice problems don’t naturally support DH-style operations
 - No efficient “commutative” group operation like $g^{ab} = g^{ba}$
 - Isogenies do support DH-style, but have other issues
- **NIST’s choice (rumored):**
 - KEMs work for all mathematical foundations
 - Unified framework for competition
 - Avoids favoring specific mathematical structures

X-Wing: An optimized hybrid KEM

- **What is X-Wing?**
 - Purpose-built hybrid KEM combining X25519 and ML-KEM-768^a
 - Designed to be “the sensible choice for most applications”
- **Efficiency gains:**
 - Tailored construction beats generic KEM combinators
 - Takes advantage of specific properties of X25519 and ML-KEM-768
- **Security guarantees:**
 - Secure if **either** X25519 OR ML-KEM-768 remains secure
 - Classical security: IND-CCA if strong DH holds (proven in ROM)
 - Post-quantum security: IND-CCA if ML-KEM-768 is IND-CCA (standard model)
- Concrete choices enable both optimizations and stronger proofs!

^a<https://appliedcryptography.page/paper/#xwing-hybrid>

X-Wing Design

X-Wing private key (2464 bytes):

ML-KEM-768 private key (2400 bytes)	X25519 private key (32 bytes)	X25519 public key (32 bytes)
--	----------------------------------	---------------------------------

X-Wing public key (1216 bytes):

ML-KEM-768 public key (1184 bytes)	X25519 public key (32 bytes)
---------------------------------------	---------------------------------

X-Wing ciphertext (1120 bytes):

ML-KEM-768 ciphertext (1088 bytes)	X25519 ciphertext (32 bytes)
---------------------------------------	---------------------------------

X-Wing shared key (32 bytes):

SHA3-256	$\left(\begin{array}{ c c c c c } \hline \backslash . / & \text{ML-KEM-768} & \text{X25519} & \text{X25519} & \text{X25519} \\ \hline / \sim \backslash & \text{shared key} & \text{shared key} & \text{ciphertext} & \text{public key} \\ \hline (6 \text{ bytes}) & (32 \text{ bytes}) & (32 \text{ bytes}) & (32 \text{ bytes}) & (32 \text{ bytes}) \\ \hline \end{array} \right)$
----------	---

The initial X-Wing label is encoded as 6-byte ASCII string "\.//~\".

Figure 1: The X-Wing KEM private key, public key, ciphertext, and shared key.

Source: <https://appliedcryptography.page/paper/#xwing-hybrid>

Section 3

Today's Post-Quantum Protocols

Subsection 3.1

Post-Quantum TLS

TLS migration timeline

- **February 2024:** PQ TLS 1.3 adoption reaches critical mass^a
- **QUIC protocol:** Uses TLS 1.3 under the hood
 - Modern transport protocol by Google
 - Built-in encryption via TLS 1.3
 - Already quantum-ready infrastructure
- **Preventing future ossification:** GREASE
 - Generate Random Extensions And Sustain Extensibility
 - Clients send unknown identifiers on purpose
 - Catches implementations that break on unknown values
 - Keeps protocols flexible for future changes

^a Much of the slides in this subsection are based on Bas Westerbaan's excellent blog post over at Cloudflare, but have been updated to account for developments in 2024 and 2025: <https://blog.cloudflare.com/pq-2024/>

ML-KEM: The NIST standard

- **Original name:** CRYSTALS-Kyber
- **International collaboration:**
 - Designers from France, Switzerland, Netherlands, Belgium, Germany, Canada, USA
 - Industry and academia partnership
- **Three security levels:**
 - ML-KEM-512: Targets AES-128 equivalent security
 - ML-KEM-768: Targets AES-192 equivalent security
 - ML-KEM-1024: Targets AES-256 equivalent security
- **First NIST PQ key encapsulation standard**

ML-KEM vs X25519 (ECDH): The numbers

Algorithm	PQ	Size (bytes)		Ops/sec	
		Client	Server	Client	Server
ML-KEM-512	✓	800	768	45,000	70,000
ML-KEM-768	✓	1,184	1,088	29,000	45,000
ML-KEM-1024	✓	1,568	1,568	20,000	30,000
X25519	×	32	32	19,000	19,000

- **Size penalty:** ML-KEM is 25–50× larger than X25519
- **Speed bonus:** ML-KEM can be faster
- **Trade-off:** Bandwidth vs computation

Hybrid key agreement: Belt and suspenders

Common deployment: X25519 + ML-KEM-768

Why ML-KEM-768?

- Not just ML-KEM-512
- Security margin against future cryptanalysis
- Lattice crypto is newer than elliptic curves
- Conservative choice

Why keep X25519?

- Hedge against ML-KEM break
- Protection against implementation bugs
- Example: KyberSlash timing attack
- Proven track record

Result: Security of both, vulnerable only if both break

Early browser experiments

- **2016:** Google's first PQ experiment (same year as NIST competition)
- **2018:** Cloudflare + Google joint experiment
 - CECQP2: X25519 + NTRU-HRSS (lattice-based)
 - CECQP2b: X25519 + SIKE (isogeny-based)
- **Results:**
 - NTRU-HRSS: Good performance, similar to ML-KEM
 - SIKE: Small keys but slow, **completely broken in 2022**
 - Good argument for hybrid cryptosystems (like X-Wing^a) that combine both classical and PQ crypto!
 - Handshake times nearly identical to classical crypto
- **But:** Compatibility issues emerged...

^a<https://appliedcryptography.page/paper/#xwing-hybrid>

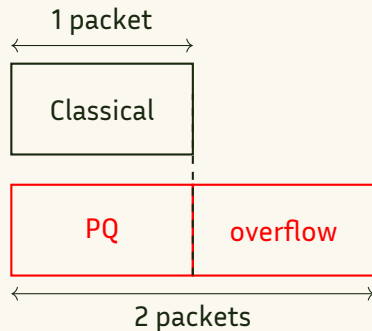
The ClientHello size problem

The ossification strikes again:

- TLS ClientHello historically fit in one packet
- PQ keyshares make it much larger
- Some middleboxes assume single-packet ClientHello
- Result: Broken connections

Chrome's response:

- Kept PQ experiment at low rate
- Worked with vendors to fix issues
- 5-year delay in deployment



Current deployment status (2025)

Browsers have begun implementing PQ TLS:

- Started with Chrome, now others following
- In total, now 38% of all TLS connections^a

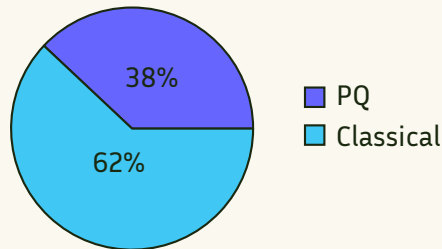
Other browsers:

- Firefox: since mid-2024
- Edge, Brave: based on Chrome, so followed it
- Safari: timeline announced at WWDC 2025^b for iOS 26, macOS 26, etc.

^a<https://radar.cloudflare.com/adoption-and-usage#post-quantum-encryption-adoption>

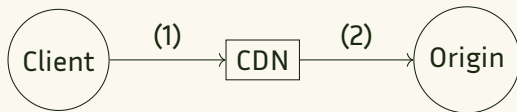
^b<https://support.apple.com/en-us/122756>

PQ Adoption



Beyond browsers: Origin connections

The full connection path:



Two approaches for CDN-to-origin PQ:

- **Fast way:** Send PQ keyshare immediately
 - Like Chrome's approach
 - 0.34% connection failures observed
- **Safe way:** Wait for HelloRetryRequest
 - Extra round trip
 - 100% compatibility

Failure causes: Large ClientHello + incorrect HelloRetryRequest handling

Lessons from PQ TLS deployment

- **Ossification is real:**
 - Assumptions about packet sizes break deployments
 - GREASE helps but isn't perfect
 - Takes years to fix ecosystem
- **Hybrid approach is prudent:**
 - Protects against algorithm breaks
 - Protects against implementation bugs
 - Small performance cost for peace of mind
- **Deployment is gradual:**
 - Started in 2016, still ongoing in 2025
 - Requires coordination across entire ecosystem
 - But progress is accelerating

Section 3

Today's Post-Quantum Protocols

Subsection 3.2

Post-Quantum Secure Messaging

Signal: PQXDH for initial key agreement

- **PQXDH:** Post-Quantum Extended Diffie-Hellman (2023)
- **Extends X3DH:** Signal's existing handshake protocol
- **Design goals:**
 - Protect against store-now-decrypt-later (SNDL) attacks
 - No security loss compared to X3DH
 - Minimal changes to existing protocol
 - Efficient for Signal's usage patterns
- **Key insight:** Add PQ-KEM alongside existing DH operations
 - Not replacing, but augmenting classical crypto
 - Hybrid approach for defense in depth

How PQXDH works

X3DH (classical):

- Multiple DH computations
- Identity keys (IK)
- Signed pre-keys (SPK)
- One-time pre-keys (OPK)
- Ephemeral keys (EK)

PQXDH additions:

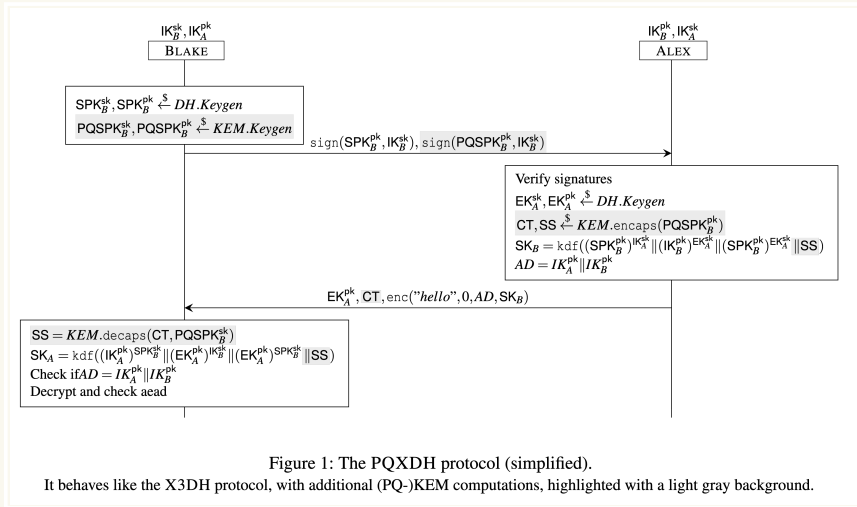
- + KEM key pairs (PQSPK)
- + KEM encapsulation
- + Shared secret in KDF

Protocol flow:

1. Bob generates KEM keys
2. Bob signs and uploads them
3. Alice encapsulates to Bob's KEM key
4. Both derive session key using:
 - DH shared secrets
 - + KEM shared secret

Result: Key secure if either DH or KEM remains unbroken

How PQXDH works



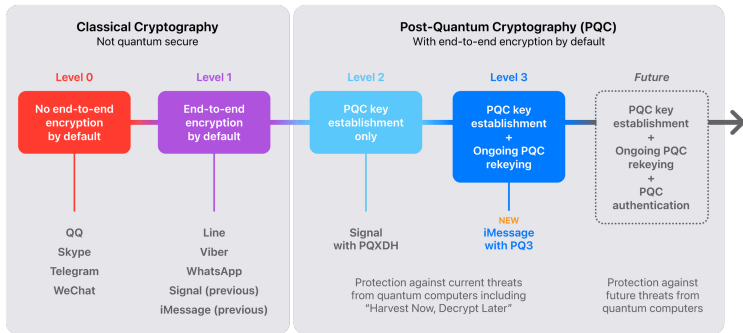
Source: <https://appliedcryptography.page/paper/#pqxdh-analysis>

Lessons from PQXDH deployment

- **Formal verification revealed issues:**
 - Public key confusion attack (DH vs KEM keys)
 - KEM re-encapsulation vulnerability
 - Fixed in specification version 2
- **Implementation considerations:**
 - Kyber1024 chosen for conservative security
 - Key format prevents confusion attacks
 - Careful integration with existing code
- **Deployment status:**
 - Already live for millions of users
 - Transparent upgrade from X3DH
 - Sets precedent for incremental PQ migration

Apple iMessage: PQ3

Quantum-Secure Cryptography in Messaging Apps



Note: This comparison evaluates only the cryptographic aspect of messaging security, and therefore focuses on end-to-end encryption and quantum security. Such a comparison doesn't include automatic key verification, which we believe is a critical protection for modern messaging apps. As of the time of this writing, only iMessage and WhatsApp provide automatic key verification. The iMessage implementation, called Contact Key Verification, is the state of the art – it provides the broadest automatic protections and applies across all of a user's devices.

Source: Apple Security Engineering and Architecture (SEAR)

iMessage PQ3: Post-quantum double ratchet

- **PQ3:** Apple's post-quantum secure messaging protocol (2024)
- **Hybrid approach:** Combines classical (ECDH) and post-quantum (ML-KEM) cryptography
- **Double ratchet construction** (like Signal):
 - Outer ratchet: Public-key ratchet (conversation direction changes)
 - Inner ratchet: Symmetric ratchet (same sender keeps sending)
- **Key innovation:** ML-KEM integrated into both initialization AND ratcheting
 - Provides post-compromise security against quantum adversaries
 - At the time, Signal only used PQ crypto in setup (PQXDH), not in ratcheting
- **Security guarantee:** Must break BOTH classical and PQ crypto to compromise

Apple iMessage PQ3: High-level overview

Device-to-device messaging:^a

- Asynchronous communication
- Either party can send anytime
- Works when peer is offline

Key components:

- Apple IDentity Service (IDS)
- Long-term identity keys
- Pre-key material
- Double ratchet + PQ

Session establishment:

1. Query IDS for peer's pre-keys
2. Derive initial keys
3. Send first encrypted message
4. Peer verifies and derives keys
5. Session established

Two ratchet types:

- **Symmetric:** Same direction
- **Public-key:** Direction change

^a<https://appliedcryptography.page/paper/#pq3-analysis>

PQ3: Hybrid cryptographic construction

Key material used:

- **Classical:** ECDH key pairs
 - Ephemeral keys (per session)
 - Pre-keys (for async start)
- **Post-quantum:** ML-KEM key pairs
 - ML-KEM-768 (ephemeral)
 - ML-KEM-1024 (pre-keys)

Root key derivation:

- Previous root key (or 0x0)
- ECDH shared secret
- KEM shared secret (or 0x0)
- Combined via HKDF

The ratcheting mechanism:

- **Symmetric ratchet:**
 - Chain key $\rightarrow$ Message key
 - Chain key $\rightarrow$ New chain key
 - No root key update
- **Public-key ratchet:**
 - New ECDH ephemeral keys
 - Sometimes: New KEM keys
 - Update root key
 - Derive new chain key

KEM refresh heuristic:

- Every ≈ 50 messages, OR
- At least once per week

PQ3: Security guarantees

- **Hybrid construction benefits:**
 - Secure if EITHER classical OR PQ remains unbroken
 - At minimum: Classical security level
 - Plus: Protection against quantum adversaries
- **Post-compromise security:**
 - Regular KEM ratcheting provides PQ healing
 - Even if KEM shared secret compromised
 - Next KEM refresh restores security
 - Works against “harvest now, decrypt later”
- **Forward secrecy:**
 - Per-message keys from chain ratchet
 - Delete old keys immediately
 - Past messages safe even if current key leaked

PQ3's bandwidth challenge: Ratcheting trade-offs

The KEM size problem:

- ML-KEM-768: 1,184 bytes sent
- ML-KEM-1024: 1,568 bytes sent
- Compare to X25519: 32 bytes!

PQ3's solution:

- Can't send new KEM key with every message
- Heuristic: New KEM key every ≈ 50 messages
- Or at least once per week
- Trade-off: Less frequent PQ ratcheting

Signal (classical):

- Small DH keys (32 bytes)
- Can ratchet with every message
- Immediate post-compromise recovery

PQ3 (post-quantum):

- Large KEM keys (1KB+)
- Ratchets every ≈ 50 messages
- Delayed post-compromise recovery
- But quantum-resistant!

Signal's Triple Ratchet

- **The problem with PQXDH:**
 - Only initial handshake is PQ-secure
 - No post-quantum post-compromise security (PCS)
- **Signal's new approach:**
 - Triple Ratchet protocol (2025)^a
 - Bandwidth-efficient hybrid-secure messaging
 - Addresses PQ3's communication inefficiencies
 - Actively being evaluated by Signal team
- **Key innovation:** Three parallel ratchets
 1. Classical public ratchet (DH-based)
 2. Post-quantum public ratchet (lattice-based)
 3. Symmetric ratchet (as before)

^a<https://appliedcryptography.page/paper/#triple-ratchet>

The erasure code innovation

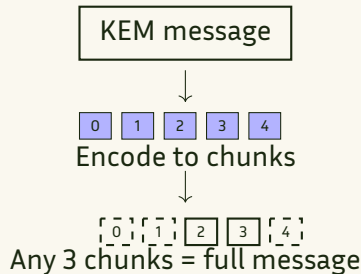
PQ3's problem:

- Sends full Kyber key every ≈ 50 messages
- If recipient offline: resends 2272 bytes repeatedly!
- Worst case: 50 $\times$ overhead

Triple Ratchet's solution:

- Split KEM message using erasure codes
- Send one chunk per message
- Recipient reconstructs after any 50 chunks
- No repeated full messages!

Erasure Code Example



Katana: Optimized lattice RKEM

- **Ratcheting KEM (RKEM):** KEM where ciphertext also serves as next public key
 - Like how Signal reuses g^b for both encryption and next public key
 - Saves bandwidth by dual-purposing values
- **Previous attempts had security flaws:**
 - Direct lattice translations of Signal's approach were broken
 - Required careful fix using hint-MLWE assumption
- **Katana:** New RKEM based on Kyber
 - 37% smaller than naive Kyber approach
 - At 192-bit security: 1,416 bytes vs 2,272 bytes
 - Provably secure under hint-MLWE

Triple Ratchet efficiency gains

Protocol	Balanced ($p=0$)	Typing ($p=0.5$)	Offline ($p=0.9$)
PQ3	6,488 B	11,176 B	48,680 B
TR + Kyber	9,000 B	9,540 B	13,860 B
TR + Katana	6,500 B	6,890 B	10,010 B

- p : Probability sender continues without waiting for response
- **Balanced**: TR competitive even in best case for PQ3
- **Realistic scenarios**: TR dramatically better (up to 5× improvement)
- **Combines**: Erasure coding + efficient Katana RKEM

Understanding the communication cost table

Key parameter: p

- Probability sender continues without receiving reply
- $p = 0$: Perfectly balanced (sender waits for reply)
- $p = 0.5$: Realistic (typing indicators, read receipts)
- $p = 0.9$: Offline device (laptop turned off)

Why PQ3 costs explode:

- Sends full Kyber-768 (2,272B) repeatedly
- Until acknowledgment received
- High p = many retransmissions

Why TR stays efficient:

- Erasure codes spread data across messages
- Each message carries one chunk
- No repeated full KEM messages
- Works even with offline devices

Real-world impact:

- Typing indicators common
- Linked devices often offline
- $p = 0.9$ realistic for Signal
- TR: 5× better than PQ3

Triple Ratchet: Technical challenges

- **Desynchronized ratchets:**
 - Classical and PQ ratchets advance at different speeds
 - Need separate root keys for each
 - Carefully combine for final message key
- **Delayed key usage:**
 - Can't use new PQ key immediately
 - Must wait for recipient to decode enough chunks
 - Protocol tracks acknowledgments carefully
- **Security analysis:**
 - Different epoch functions for classical vs PQ
 - PQ healing takes 3 epochs (vs 2 for classical)
 - But still provides hybrid security guarantees

Comparison: PQ3 vs Triple Ratchet

Apple PQ3:

- Simple amortization strategy
- Full KEM every ≈ 50 messages
- Good for balanced chats
- **Bad for unbalanced scenarios**

Signal Triple Ratchet:

- Sophisticated erasure coding
- Chunks spread across messages
- Efficient in all scenarios
- More complex implementation

Key insight: Different trade-offs for different use cases
Both are significant advances in PQ messaging!

Section 3

Today's Post-Quantum Protocols

Subsection 3.3

Post-Quantum SSH

OpenSSH: Post-quantum by default

- **OpenSSH leading the way:** PQ key agreement by default since 2022
- **Timeline of PQ adoption:**
 - **OpenSSH 9.0 (2022):** First PQ by default
 - Algorithm: `sntrup761x25519-sha512`
 - Hybrid approach (NTRU + X25519)
 - **OpenSSH 9.9 (2024):** Added ML-KEM support
 - Algorithm: `mlkem768x25519-sha256`
 - Based on NIST standard
 - **OpenSSH 10.0:** ML-KEM becomes default
 - **OpenSSH 10.1:** Warns on non-PQ connections
- **The issue:** SSH particularly vulnerable to SNDL
 - Entire session privacy depends on key agreement
 - Attackers can record SSH sessions today
 - Decrypt everything once QC arrives

OpenSSH's aggressive stance

The warning (10.1+):

*** WARNING: connection is not using a post-quantum key exchange algorithm.*

*** This session may be vulnerable to "store now, decrypt later" attacks.*

Why this matters:

- Pushes ecosystem forward
- Makes PQ the norm, not exception

SSH's unique position:^a

- Often used for critical infrastructure
- Long-lived connections
- High-value targets (admin access)
- Perfect for SNDL attacks

Deployment advantage:

- Simpler than TLS
- No certificate chains
- Easier to upgrade

^a<https://www.openssh.com/pq.html>

Section 3

Today's Post-Quantum Protocols

Subsection 3.4

Post-Quantum Cryptocurrency

Cryptocurrency's unique quantum vulnerability

- **Foundation:** Bitcoin, Ethereum, and most cryptocurrencies rely on ECDSA over secp256k1
- **Shor's algorithm breaks this:** Recovers private keys from public keys in polynomial time
- **Unlike TLS:** Blockchain keys are often exposed *permanently* on-chain
 - **P2PK scripts:** Public key stored directly (~1.7M BTC)
 - **Address reuse:** Public key exposed after first spend
 - **Taproot (P2TR):** Stores tweaked public key in script — a quantum security regression
- **Total:** ~6.7 million BTC in quantum-vulnerable addresses¹

Three types of quantum attacks on blockchains

Attack	Window	Architecture	Target
On-spend	Seconds to minutes	Fast-clock only	Mempool transactions
At-rest	Days or more	Any CRQC	Exposed on-chain keys
On-setup	One-time (offline)	Any CRQC	Protocol parameters

- **On-spend:** Derive private key while transaction sits in the mempool
 - ~9 min on fast-clock CRQC vs. Bitcoin's 10-min block time (~41% success)
- **At-rest:** Target keys already visible on-chain (any CRQC suffices)
- **On-setup:** Attack fixed protocol parameters to create a *reusable classical backdoor*
 - Example: Recovering KZG trusted setup "toxic waste" on Ethereum

Beyond Bitcoin: Ethereum's expanded attack surface

Category	Mechanism	At Risk
Account	ECDSA (key exposed on tx)	~20.5M ETH (top 1,000)
Admin	Unrotated admin keys	~\$200B stablecoins/RWAs
Code	No PQC EVM precompiles	All L2s, bridges
Consensus	BLS12-381 (PoS)	~37M ETH staked
Data Availability	KZG trusted setup	Universal backdoor risk

- **On-setup attack on KZG:** One-time quantum computation recovers trusted setup “toxic waste,” creating a *persistent classical backdoor* for forging data availability proofs
- **Shorter block time** (12s) makes on-spend attacks harder than on Bitcoin

Dormant assets and the migration race

The dormant assets problem:

- ~2.3M BTC in dormant quantum-vulnerable addresses
- Includes ~1M BTC attributed to Satoshi Nakamoto
- **Fixed target:** Tens to hundreds of billions of dollars

Policy options:

- Do nothing (allow quantum seizure)
- Burn (render unspendable)
- Hourglass (rate-limit spending)
- Digital salvage sidechain

The migration race:

- Bitcoin: Migration would take months at current transaction rate
- Ethereum: EIP-7932 proposes PQ precompiles
- Margin for error “increasingly narrow”

“It is conceivable that the existence of early CRQCs may first be detected on the blockchain rather than announced.”

Responsible disclosure via zero-knowledge proofs

- **The dilemma:** How to warn about quantum vulnerabilities without revealing attack circuits?
- **Google's approach^a:** Use a ZKP to prove the resource estimates
- **What they prove:**
 1. Possession of circuit that computes EC point addition on secp256k1
 2. Circuit satisfies the claimed resource bounds

^a<https://research.google/blog/safeguarding-cryptocurrency-by-disclosing-quantum-vulnerabilities-responsibly/>

Security:

- If the circuit fails on $>1\%$ of inputs: $(0.99)^{9024} \approx 2^{-130}$ chance of passing
- 128-bit cryptographic confidence
- **Irony:** The Groth16 SNARK itself relies on pairing-friendly elliptic curves
- Its soundness is vulnerable to the very quantum attacks it validates!

Section 4

Google's Threat Model for Post-Quantum Cryptography

Google as a PQ case study

- **Why look at Google?**
 - Major cloud provider with diverse crypto needs
 - Building their own quantum computer (Google Quantum AI)
 - Public about their PQ migration strategy
- **Google's approach:** Risk-based prioritization
 - Not all crypto needs equal urgency
 - Different algorithms for different use cases
 - Timeline-driven deployment
- **Let's examine their threat model from 2024...**^a

^a<https://bughunters.google.com/blog/5108747984306176/google-s-threat-model-for-post-quantum-cryptography>

Google's prioritization framework

Four key considerations for quantum threats:

1. **Attack feasibility:** How realistic is the quantum attack?
2. **Store-now-decrypt-later:** Is data at risk today?
3. **Key lifetime:** Do public keys last for decades?
4. **Redesign needs:** Will systems need major changes?

Not all cryptography is equally threatened

Google's crypto taxonomy

By quantum vulnerability:

1. Asymmetric encryption/KEM

- Broken by Shor's algorithm
- **Vulnerable to SNDL (Store-Now-Decrypt-Later)**

2. Digital signatures

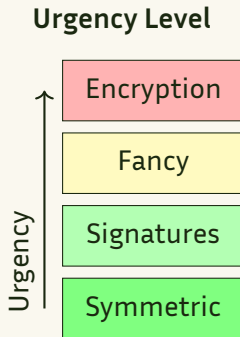
- Broken by Shor's algorithm
- **Not vulnerable to SNDL (Store-Now-Decrypt-Later)**

3. "Fancy" crypto

- Blind signatures, OPRF, etc.
- Case-by-case assessment needed

4. Symmetric crypto

- **Practically safe from QC**
- Grover's gives "only" $\sqrt{n}$ speedup



Google's use case analysis

Use Case	SNDL Risk	Urgency	Challenge
Encryption in Transit (TLS, SSH...)	High	Critical	Deployment scale
Firmware Signatures (Secure boot)	Medium	High	Can't update keys
Software Signatures	Low	Medium	Size flexibility
PKI/Certificates	Low	Medium	Chain size explosion
Stateless Tokens (JWT)	Low	Low	4KB cookie limit

Key insight: Different problems need different solutions

Google's algorithm choices

Not one-size-fits-all approach:

- **Encryption in transit:** ML-KEM-768 + X25519/P256
 - Most urgent due to SNDL
 - Ephemeral keys make deployment easier
- **Firmware signatures:** SLH-DSA (SPHINCS+)
 - Can't change keys once burned to silicon
 - Hash-based = most conservative choice
 - Size less critical than security
- **Software signatures:** ML-DSA (Dilithium) + classical
 - Can update keys if needed
 - Signing large binaries = performance matters

The PKI size explosion problem

Certificate chains today:

- Root CA: $\approx 1\text{KB}$
- Intermediate: $\approx 1\text{KB}$
- Leaf cert: $\approx 1\text{KB}$
- **Total: $\approx 3\text{KB}$**

With PQ signatures:

- Each cert: $> 5\text{KB}$
- 3-cert chain: $> 15\text{KB}$
- **5 $\times$ increase!**

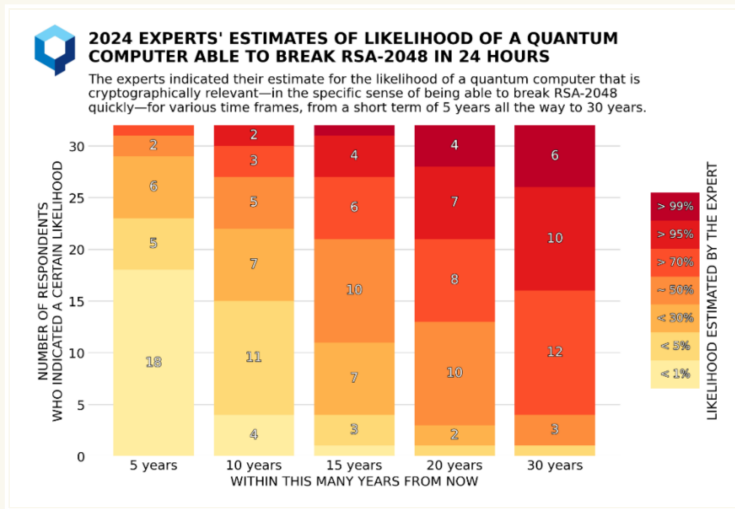
Why this matters:

- Some devices fail at 10-30KB packets
- TLS handshake performance hit
- Mobile data costs

Google's view:

"This problem might be fixable, but a severe performance penalty remains"

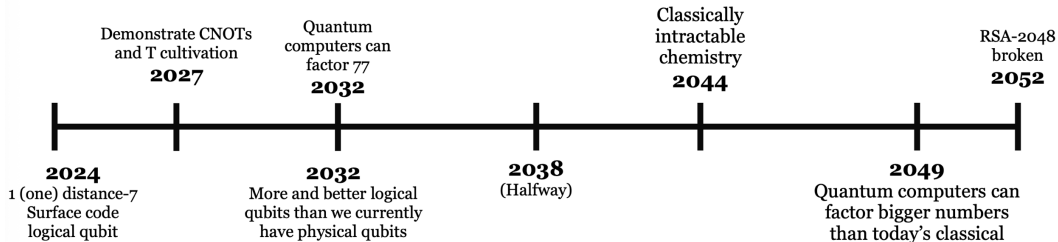
Timeline: when will quantum computing arrive?



Source: Michele Mosca and Marco Piani, Quantum Threat Timeline Report, Global Risk Institute, 2024.

Timeline: according to Sam Jaques

Assume qubits double every 1.5 years and error rate approaches 10^{-3} :



Source: Sam Jaques

Threat actors: Who gets QC first?

1. Nation states

- Most likely first adopters
- Will use deniability to hide capability
- Target: High-value stored data
- Focus on static keys (PKI)

2. Insider threats

- Google builds quantum computers
- Crown jewel protection critical

3. Financially motivated

- Only when QC becomes cheap
- Target: Unmigrated systems
- Ransomware potential

Key insight: Different actors = different timelines

Regulatory pressure

- **US Government:**
 - Executive orders requiring PQ migration
 - CNSA 2.0: Aggressive timelines for government systems
 - FIPS standards coming soon
- **Europe:**
 - BSI (Germany) and ANSSI (France) active
 - Requesting PQ roadmaps from vendors
- **Impact on Google:**
 - Government contracts require compliance
 - Accelerates migration timeline
 - Can't wait for perfect solutions

Google's lessons for the industry

1. Prioritize by risk, not uniformly

- Encryption in transit: Do it now
- Signatures: You have more time

2. Match algorithms to use cases

- No universal “best” PQ algorithm
- Consider size, speed, and security trade-offs

3. Hybrid everything

- PQ algorithms less mature than classical
- Defense against both classical and quantum attacks

4. Start experimenting now

- PKI and tokens need creative solutions
- Industry collaboration essential

Section 5

How Things Can Go Wrong

Reality check: Post-quantum isn't perfect

- **Post-quantum $\neq$ unbreakable**
- **Fundamental challenge:** These schemes are newer
 - RSA: 40+ years of cryptanalysis
 - Elliptic curves: 30+ years
 - Most PQ schemes: 10-20 years
- **Limited understanding brings risks:**
 - Security levels harder to quantify
 - Implementation pitfalls less understood
 - Potential for unexpected attacks
- Let's explore what can go wrong...

Section 5

How Things Can Go Wrong

Subsection 5.1

Unclear Security Levels

The deceptive strength problem

- **Some schemes look incredibly strong on paper...**
- **Example:** McEliece (hardness of decoding a linear code with insufficient information)
 - In the worst case, some argue could be proven NP-hard
 - Sounds amazing: NP-hard = practically unbreakable!
 - But cryptographic problems are based on the average case, not the worst case...
 - And the claim of NP-hardness even here is debated...^a
- **Real-world complications:**
 - Security proofs only work for certain parameters
 - Practical implementations use different parameters
 - The gap between theory and practice can be exploited

^aEurocrypt 2025 paper found a subexponential distinguisher for McEliece: <https://appliedcryptography.page/paper/#syzygy-distinguisher>

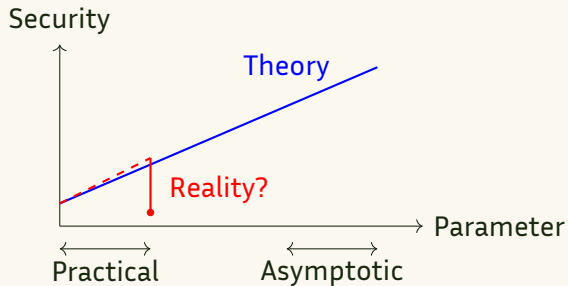
Asymptotic vs. concrete security

Asymptotic security:

- “Secure as $n \rightarrow \infty$ ”
- Mathematical proofs assume large parameters
- Example: “Secure for lattice dimension $> 10^6$ ”

Concrete security:

- What we actually use: $n = 512$
- May be far from asymptotic regime
- Security at practical parameters unclear



The unknown attack landscape

- **With RSA:** We know strong attacks
 - Number Field Sieve for factoring
 - Decades of optimization
 - Can estimate security precisely
- **With lattice-based crypto:** Still discovering attacks
 - New attack improvements regularly published
 - Hard to predict future improvements
 - Security estimates keep changing
- **Example:** NIST Round 2 eliminations
 - Several “secure” schemes broken during competition
 - Some had security proofs!
 - Proofs were correct, but assumptions weren’t

Section 5

How Things Can Go Wrong

Subsection 5.2

When Quantum Computers Arrive

Breaking news from 2052...

- **The nightmare scenario:** Large quantum computer suddenly exists
- **All classical crypto broken overnight**
- **Common misconception:** “Signatures are recoverable, encryption isn’t”
- **Reality:** It depends on where the crypto lives...

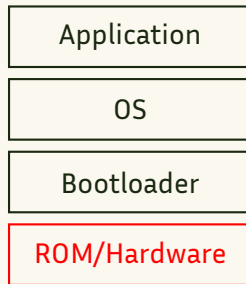
Digital signatures: Not always recoverable

Software signatures:

- Generate new PQ keys
- Re-sign and re-deploy
- Revoke old keys
- Recoverable

Hardware signatures:

- Keys burned in ROM/fuses
- Can't update silicon
- Foundation of all security
- Permanently broken



← Immutable!

The hardware root of trust problem

- **Every device starts with hardware-based security:**
 - ROM code executes first
 - Public keys in fuses/OTP memory
 - Hardware crypto accelerators
 - Debug unlock mechanisms
- **Secure boot chain:** Each stage verifies the next
 - ROM verifies bootloader signature
 - Bootloader verifies OS signature
 - OS verifies application signatures
- **If ROM keys are broken:** Entire chain collapses
 - Can install any bootloader
 - Bypass all security checks
 - Device permanently compromised

Long-lived infrastructure: The real disaster

Consumer devices:

- Phones: 2-3 year lifecycle
- Laptops: 3-5 years
- Painful but manageable

Critical infrastructure:

- Power plants: 30-50 years?
- Aircraft: 20-30 years?
- Satellites: 10-15 years?
- Military systems: 20-40 years?
- Replacement = \$billions

The \$20 chip problem:

- Cheap secure element
- Deeply embedded everywhere
- Controls \$1M+ equipment
- Can't extract and replace

Example: Car ECUs

- Cars may have multiple ECUs
- Each has secure boot
- Cars last 10-20 years

Encryption vs signatures: Both can be catastrophic

Attack Type	Encryption	Signatures
Past data	Exposed	Historical record damaged
Future security	Can use PQ crypto	HW broken
Recovery cost	Re-encrypt (if possible)	Replace hardware
Timeline	Immediate exposure	Permanent vulnerability
Worst case	Secrets revealed	Infrastructure compromised

Reality: Both are potential disasters with different characteristics

Real-world impact of broken hardware signatures

- **Secure boot bypass:** Install any firmware
 - Persistent malware
 - Undetectable rootkits
 - Complete device control
- **Debug interface unlock:** Full hardware access
 - Extract all secrets
 - Modify hardware behavior
 - Bypass all protections
- **Supply chain attacks:** Counterfeit components
 - Fake firmware updates
 - Trojan hardware accepted as genuine
 - No way to verify authenticity

The true cost of “Q-day” for signatures

Software systems:

- Cost: Engineering time
- Timeline: Weeks to months
- Impact: Service disruption

Hardware systems:

- Cost: Full replacement
- Timeline: Years to decades
- Impact: Systemic failure

Cost estimates:

- Power grid: Billions?
- Military fleet: Billions?
- Global auto industry: Trillions?
- Satellite constellation: Who knows

Bottom line: Potentially massive cost

Key agreement: It's complicated

Simple Diffie-Hellman:

- Alice: g^a , Bob: g^b
- Shared: g^{ab}
- Quantum computer recovers a or b
- All past communications compromised

Modern protocols (Signal):

- New DH for each message
- Keys depend on message history
- Even with QC, need full conversation
- Much harder to break retroactively

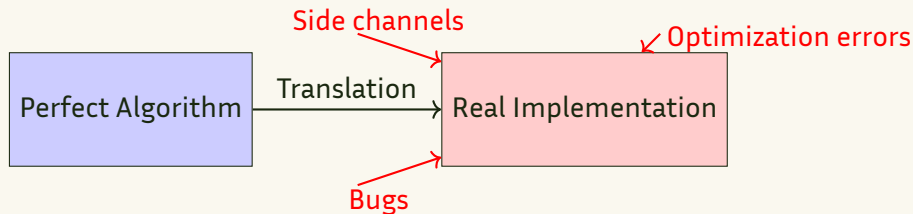
Section 5

How Things Can Go Wrong

Subsection 5.3

Implementation Vulnerabilities

The implementation gap



- **Theory:** Algorithm is perfectly secure
- **Practice:** Implementation has vulnerabilities
- **The gap:** Where security breaks down

A cautionary tale: The goto fail bug

Actual bug in Apple's SSL/TLS (2014):

```
if ((err = SSLHashSHA1.update(&hashCtx,  
    &signedParams)) != 0)  
    goto fail;  
goto fail; // <-- Extra goto!  
// This code always skipped:  
if ((err = SSLHashSHA1.final(&hashCtx,  
    &hashOut)) != 0)  
    goto fail;
```

Impact:

- Signature verification bypassed
- Any certificate accepted
- Complete security failure
- From one duplicated line!

Lesson: Even “unbreakable” crypto fails with implementation bugs

Side-channel attacks: The hidden threat

- **What are side channels?**
 - Information leaked during computation
 - Not through the algorithm itself
 - Through physical effects
- **Common side channels:**
 - **Timing:** Operations take different times
 - **Power:** Different operations use different power
 - **Cache:** Memory access patterns reveal secrets
 - **Electromagnetic:** EM emissions during computation
- **Why PQ crypto is vulnerable:** Complex mathematical operations
 - Matrix multiplications (lattice-based)
 - Polynomial operations (code-based)
 - Many conditional branches

KyberSlash: A real-world PQ vulnerability

- **What:** Timing attack on Kyber implementations^a (Dec 2023)
- **Affected:** Multiple “production-ready” Kyber libraries
- **The vulnerability:**
 - Division operations in decoding took variable time
 - Time depended on secret polynomial coefficients
 - Attacker measures decryption times
 - Recovers secret key within a couple million queries
- **Impact:** Complete key recovery in “secure” implementations!
- **Lesson:** Even NIST-standardized PQ crypto had critical flaws
 - Years of review missed this
 - “Constant-time” code wasn’t
 - Shows immaturity of PQ implementations

^a<https://appliedcryptography.page/paper/#kyber-slash>

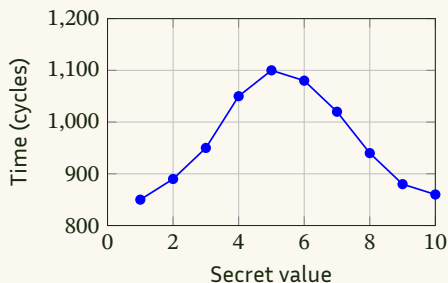
Timing attacks on post-quantum cryptography

Example: Gaussian sampling in lattices

- Need to sample from discrete Gaussian
- Rejection sampling: variable time
- Time depends on secret values!

Attack scenario:

1. Measure decryption times
2. Infer error vector properties
3. Recover secret key



Timing reveals secret information!

The PQ paradox

- PQ schemes may initially be LESS secure than classical!
- **Why?**
 - RSA/ECC: 40 years of implementation hardening
 - Constant-time implementations well understood
 - Side-channel protections mature
- **Post-quantum crypto:**
 - New implementations = new bugs
 - Complex operations = more side channels
 - Less implementation experience
- **Timeline of maturity:**
 - Years 1-5: Discovering vulnerabilities
 - Years 5-10: Developing countermeasures
 - Years 10+: Mature, hardened implementations

Lessons learned

- **Post-quantum $\neq$ post-security-concerns**
- **Key challenges remain:**
 1. Unclear concrete security levels
 2. Implementation vulnerabilities
 3. Side-channel attacks
 4. The “store now, decrypt later” threat
- **Best practices:**
 - Use conservative parameters
 - Implement hybrid solutions (classical + PQ)
 - Invest in secure implementations
 - Plan for crypto-agility



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.6: Post-Quantum Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>